Fase 4

DIVIDIR PARA PROGRAMAR

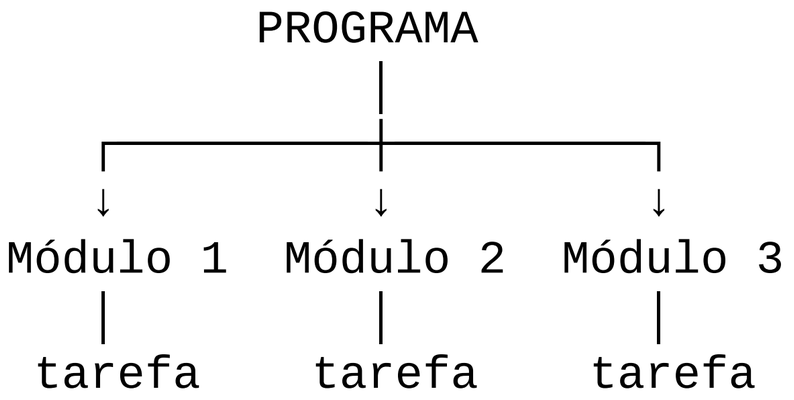


APRESENTADO POR:
LORENA EDUARDA

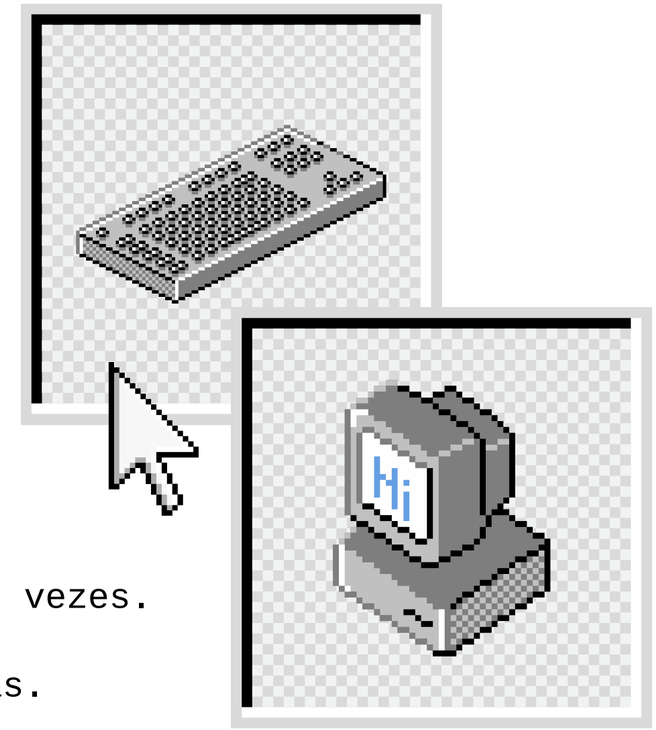
Start

O QUE É MODULARIZAÇÃO

Modularização é dividir um programa em partes menores, chamadas de módulos.
Cada módulo fica responsável por uma tarefa específica.



POR QUE MODULARIZAR?



Organização
Código dividido em partes menores.

Reutilização
Podemos utilizar o mesmo módulo várias vezes.

Manutenção
Facilita encontrar e corrigir problemas.

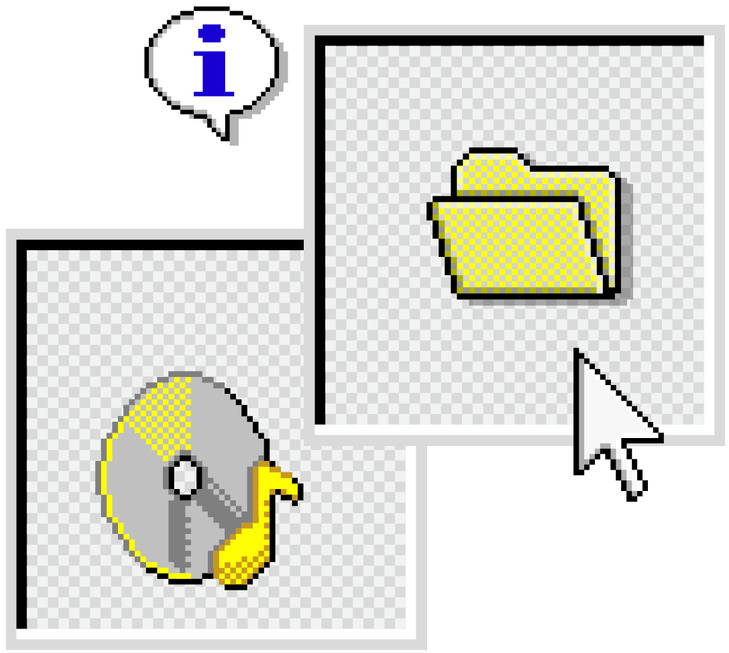
Legibilidade
O código fica mais fácil de entender.

```
printf("===== MENU =====\n");
printf("1 - Cadastrar\n");
printf("2 - Consultar\n");
printf("3 - Sair\n");

printf("===== MENU =====\n");
printf("1 - Cadastrar\n");
printf("2 - Consultar\n");
printf("3 - Sair\n");
```

O código está repetido.
E se precisarmos alterar o menu?





```
void mostrarMenu() {
    printf("===== MENU =====\n");
    printf("1 - Cadastrar\n");
    printf("2 - Consultar\n");
    printf("3 - Sair\n");
}

Agora podemos chamar:
mostrarMenu();
```

TRANSFORMANDO EM UM BLOCO REUTILIZÁVEL



PROCEDIMENTOS

Um procedimento é um bloco de código criado para realizar uma tarefa específica.
Em C, podemos representá-lo com void.

```
void mostrarMensagem() {
    printf("Olá, aluno!\n");
}

Para executar:
mostrarMensagem();
```



CRIAR ≠ EXECUTAR

```
void mostrarMensagem() {
    printf("Olá, aluno!\n");
}
```

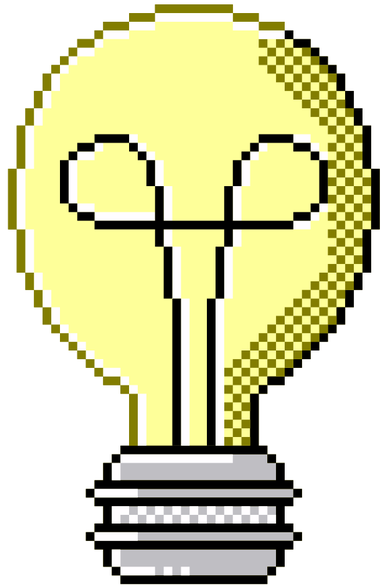
⬇️

O procedimento foi criado.

Para executá-lo:
`mostrarMensagem();`

⬇️

O procedimento é chamado.



```
int main() {


mostrarMensagem();

mostrarMensagem();

mostrarMensagem();

    return 0;
}
```




```
Resultado:
Olá, aluno!
Olá, aluno!
Olá, aluno!

Escrevemos uma vez.
Utilizamos várias vezes.
```

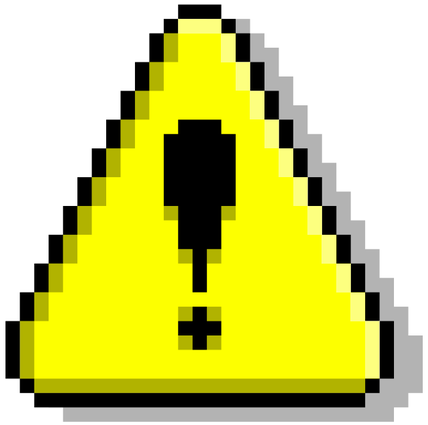
```
void mostrarDobro(int numero) {
    printf("Dobro: %d\n", numero * 2);
}

Podemos chamar:
mostrarDobro(5);
mostrarDobro(10);

Resultado:
Dobro: 10
Dobro: 20
```



PARÂMETROS



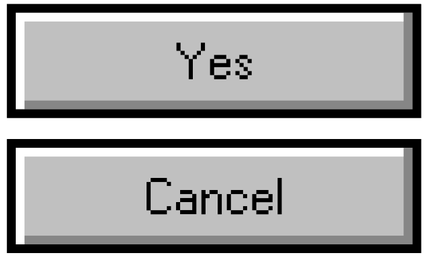


Imagine:
mostrarDobro(5);
Ele mostra o resultado.

Mas e se quisermos guardar o resultado?
resultado = dobro(5)
Precisamos de uma função que retorne um valor.



FUNÇÕES



Uma função é um bloco de código que:

- realiza uma tarefa;
- pode receber parâmetros;
- retorna um valor.

Exemplo:

```
int somar(int a, int b) {
    return a + b;
}
```

Uso:

```
int resultado = somar(5, 3);
```

Procedimento

- Pode receber parâmetros
- Não retorna um valor
- void

Função

- Realiza uma tarefa
- Pode receber parâmetros
- Retorna um valor



PROCEDIMENTO × FUNÇÃO

DESAFIO 1: GRITO DE GUERRA

[MISSÃO 1] Grito de Guerra da Guilda (Procedimento void)

Toda vez que sua equipe encontra inimigos, um alerta de combate aparece!

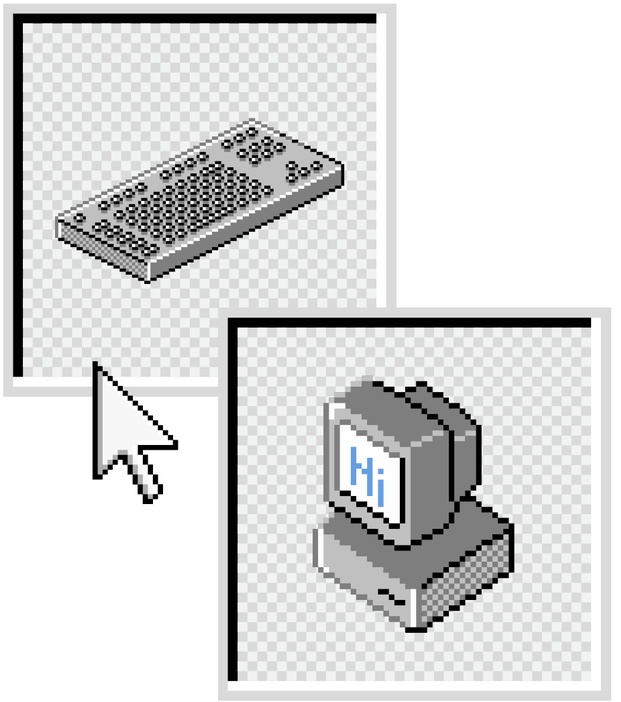
- Em vez de repetir vários printf, vamos criar um procedimento reutilizável!

O que o seu programa em C deve fazer:

1. Criar o procedimento void alertaBatalha() antes da função main().
2. Dentro dele, mostrar "=== ALERTA: INIMIGO A VISTA! ===" e "Preparem-se!".
3. Na função main(), chamar alertaBatalha(); duas vezes seguidas!
4. Observar como o bloco é executado duas vezes sem repetir código!

Exemplo de execução na tela:

```
=== ALERTA: INIMIGO A VISTA! ===   |   Preparem-se!
=== ALERTA: INIMIGO A VISTA! ===   |   Preparem-se!
```



UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

DESAFIO 2: FEITIÇO DE CURA

[MISSÃO 2] Feitiço de Cura do Mago (Procedimento com Parâmetro)

O Mago do grupo aprendeu um feitiço especial para restaurar a vida do herói!

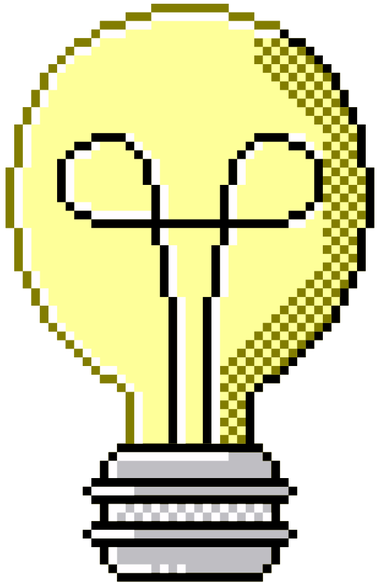
- O feitiço recebe a energia mágica gasta e cura o DOBRO desse valor em vida!

O que o seu programa em C deve fazer:

1. Criar o procedimento void usarCura(int magia) que recebe 1 parâmetro inteiro.
2. Dentro dele, mostrar na tela: printf("Cura: +%d de Vida!\n", magia * 2);
3. Na main(), ler (scanf) a energia mágica digitada pelo jogador.
4. Chamar usarCura(energia); passando o valor lido como parâmetro!

Exemplo de execução na tela:

Digite a energia magica: 15
Cura: +30 de Vida!



UTFPR
UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

DESAFIO 3: DANO NO CHEFÃO

[MISSÃO 3] Calculadora de Dano no Chefão (Função com return)

Na batalha contra o Chefão, precisamos calcular e GUARDAR o dano total causado!

- O dano total é a soma da força da arma com o bônus mágico do herói.

O que o seu programa em C deve fazer:

1. Criar a função int calcularDano(int arma, int bonus) que retorna arma + bonus.
2. Na main(), ler (scanf) a força da arma e o bônus mágico do jogador.
3. Chamar a função guardando o retorno: total = calcularDano(arma, bonus);
4. Mostrar na tela (printf) o dano total calculado pela função!

Exemplo de execução na tela:

Dano da arma: 40 | Bonus magico: 25
Dano total causado no Chefao: 65 pontos!



UTFPR
UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

GABARITO 1: GRITO DE GUERRA

CÓDIGO RESOLVIDO EM C

```
01 | #include <stdio.h>
02 | void alertaBatalha() {
03 |     printf("=== INIMIGO A VISTA! ===\n");
04 |     printf("Preparem-se!\n");
05 | }
06 | int main() {
07 |     printf("Iniciando missao...\n");
08 |     alertaBatalha();
09 |     alertaBatalha();
10 |     return 0;
11 | }
```

EXPLICAÇÃO DE CADA LINHA

-> Inclui a biblioteca padrão de entrada e saída
-> Cria um procedimento (void = não retorna valor)
-> 1a mensagem exibida quando o alerta é chamado
-> 2a mensagem exibida pelo procedimento
-> Fecha o bloco do procedimento alertaBatalha
-> Inicia a função principal onde o programa começa
-> Mostra mensagem de início da missão
-> 1a chamada: executa o procedimento na tela
-> 2a chamada: reutiliza o mesmo procedimento!
-> Indica que o programa terminou com sucesso
-> Fecha o bloco da função principal (main)

GABARITO 2: FEITIÇO DE CURA

CÓDIGO RESOLVIDO EM C

```
01 | #include <stdio.h>
02 | void usarCura(int magia) {
03 |     printf("Cura: +%d de Vida!\n", magia * 2);
04 | }
05 | int main() {
06 |     int energia;
07 |     printf("Digite a energia magica: ");
08 |     scanf("%d", &energia);
09 |     usarCura(energia);
10 |     return 0;
11 | }
```

EXPLICAÇÃO DE CADA LINHA

-> Inclui a biblioteca para usar printf e scanf
-> Cria procedimento que recebe o parâmetro 'magia'
-> Calcula o dobro da magia recebida e mostra na tela
-> Fecha o bloco do procedimento usarCura
-> Inicia a função principal do programa em C
-> Cria variável para guardar a energia digitada
-> Pede ao jogador a quantidade de energia mágica
-> Lê o número inteiro e guarda em 'energia'
-> Chama o procedimento enviando 'energia' a ele
-> Indica que o programa encerrou sem erros
-> Fecha o bloco da função principal (main)

GABARITO 3: DANO NO CHEFÃO

CÓDIGO RESOLVIDO EM C

```
01 | #include <stdio.h>
02 | int calcularDano(int arma, int bonus) {
03 |     return arma + bonus;
04 | }
05 | int main() {
06 |     int a, b, total;
07 |     printf("Dano da arma: ");
08 |     scanf("%d", &a);
09 |     printf("Bonus magico: ");
10 |     scanf("%d", &b);
11 |     total = calcularDano(a, b);
12 |     printf("Dano total: %d\n", total);
13 |     return 0;
14 | }
```

EXPLICAÇÃO DE CADA LINHA

-> Inclui a biblioteca padrão de entrada e saída
-> Cria função que recebe 2 valores e retorna um int
-> Soma os parâmetros e devolve (return) o resultado
-> Fecha o bloco da função calcularDano
-> Inicia a função principal do programa
-> Cria variáveis para a arma, o bônus e o resultado
-> Pede ao jogador o valor de ataque da arma
-> Lê o valor digitado e armazena na variável 'a'
-> Pede ao jogador o valor do bônus mágico
-> Lê o valor digitado e armazena na variável 'b'
-> Chama a função e guarda o valor retornado em 'total'
-> Mostra na tela o resultado guardado em 'total'
-> Finaliza a função principal indicando sucesso
-> Fecha o bloco da função principal (main)

OBRIGADO(A) PELA ATENÇÃO

Modularização | Procedimento | Função

